

Partie 05 — Répéter des opérations avec des boucles

INF-STAR — Programmation statistique avec R

Oskar Laverny

September 30, 2026

Aix-Marseille Université

Objectifs

Vous saurez :

- lire et écrire une boucle `for` ;
- choisir entre itérer sur des valeurs, des indices ou des noms ;
- préallouer un résultat ;
- utiliser une boucle `while` lorsque le nombre d'itérations n'est pas connu ;
- reconnaître les cas où une opération vectorisée est plus simple.

Pourquoi une boucle ?



Répéter explicitement

Supposons trois températures :

```
temp <- c(36.7, 37.2, 38.4)
```

On peut vouloir appliquer la même logique à chaque valeur.

Une boucle rend la répétition explicite.

for



```
for (valeur in sequence) {  
    # instructions  
}
```

Exemple :

```
for (x in c(2, 4, 6)) {  
    print(x^2)  
}
```

Lire une boucle pas à pas

Pour comprendre une boucle, suivez la valeur de la variable d'itération :

tour	x	x^2
1	2	4
2	4	16
3	6	36

À chaque tour, x reçoit la valeur suivante de la séquence. Après le dernier tour, l'exécution continue sous la boucle.

Itérer sur les valeurs

```
ages <- c(34, 51, 68)

for (age in ages) {
  print(age)
}
```

C'est la forme la plus lisible lorsque l'indice n'est pas nécessaire.

Itérer sur les indices

```
ages <- c(34, 51, 68)

for (i in seq_along(ages)) {
  print(ages[i])
}
```

Préférez `seq_along(x)` à `1:length(x)` : le premier se comporte correctement même si `x` est vide.

Construire un résultat : préallouer

À éviter dans une grosse boucle :

```
res <- c()
for (i in 1:10000) {
  res <- c(res, i^2)
}
```

Préférez :

```
res <- numeric(10000)
for (i in seq_along(res)) {
  res[i] <- i^2
}
```

Pourquoi préallouer ?

Si un objet grandit à chaque tour, R peut devoir réserver et recopier de la mémoire plusieurs fois.

Préallouer :

- rend l'intention explicite ;
- évite des allocations répétées ;
- améliore souvent les performances.

Nous mesurerons cet effet dans la partie benchmarking.

Boucle sur des colonnes

```
vars <- c("age", "tas", "score")

for (nom in vars) {
  print(mean(patients[[nom]], na.rm = TRUE))
}
```

`[[nom]]` permet d'accéder à une colonne dont le nom est contenu dans une variable.

```
x <- c(2, NA, 5, 8)

for (val in x) {
  if (is.na(val)) next
  print(val^2)
}
```

next passe directement à l'itération suivante.

Valeurs manquantes dans une condition

Cette condition peut échouer si `x[i]` vaut `NA` :

```
if (x[i] > 0) { ... }
```

Traitez d'abord le cas manquant :

```
if (is.na(x[i])) {  
  res[i] <- NA  
  next  
}
```

```
if (x[i] > 0) res[i] <- "positif"
```

```
for (i in 1:100) {  
  if (i^2 > 500) break  
}
```

break quitte la boucle.

while



Quand le nombre de répétitions est inconnu

```
x <- 100

while (x > 1) {
  x <- x / 2
  print(x)
}
```

La condition est vérifiée avant chaque nouvelle itération.

Attention aux boucles infinies

```
while (TRUE) {  
    # si rien ne permet de sortir : problème  
}
```

Avec `while`, identifiez toujours ce qui fait évoluer la condition ou déclenche un `break`.

Boucle ou vectorisation ?



Une boucle n'est pas « mauvaise »

Ceci est correct :

```
res <- numeric(length(ages))  
for (i in seq_along(ages)) {  
  res[i] <- ages[i] + 1  
}
```

Mais R sait déjà faire :

```
ages + 1
```

Utilisez la forme la plus claire pour le problème.

Exemple : catégorie d'IMC

Une boucle peut aider à comprendre l'algorithme :

```
categorie <- character(length(imc))

for (i in seq_along(imc)) {
  if (imc[i] < 25) {
    categorie[i] <- "< 25"
  } else {
    categorie[i] <- "25+"
  }
}
```

Puis comparez avec `ifelse()` ou `case_when()`.

Déboguer une boucle



Erreurs fréquentes

- `for (i in length(x))` ne parcourt qu'une seule valeur ;
- oublier `res[i] <-` écrase ou perd les résultats précédents ;
- utiliser `x[i]` alors que la boucle parcourt directement les valeurs ;
- oublier de traiter **NA** dans une condition ;
- oublier de faire évoluer la condition d'une boucle `while`.

Affichez temporairement `i`, la valeur courante et le résultat pour localiser le premier tour incorrect.

Une méthode de travail



Écrire une boucle progressivement

1. faites fonctionner l'opération pour **un** élément ;
2. identifiez ce qui change d'une répétition à l'autre ;
3. créez la séquence d'itération ;
4. préallouez le résultat si nécessaire ;
5. écrivez la boucle ;
6. vérifiez les premières valeurs.

À retenir



Trois formes fréquentes

```
for (x in valeurs) { ... }  
for (i in seq_along(x)) { ... }  
while (condition) { ... }
```

Les boucles sont utiles pour comprendre le contrôle du programme. Elles ne sont pas incompatibles avec la vectorisation ni avec la famille **apply*, que nous verrons plus tard.